부록 · Spring Boot — Java · Kotlin

Spring Boot Java + Kotlin JPA · DTO · Repository · Entity 연동 예제

Oracle · PostgreSQL · MariaDB 비교 및 REST API 실습

이 문서에서 고친 곳

- Java 예제가 Lombok 을 쓰는데 의존성 안내가 없어, 그대로 따라 하면 컴파일 오류가 납니다. 아래 의존성을 먼저 넣습니다.

```xml
<!-- pom.xml 의 <dependencies> 안에 -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

```kotlin
// build.gradle.kts 를 쓰는 Java 프로젝트라면
compileOnly("org.projectlombok:lombok")
annotationProcessor("org.projectlombok:lombok")
```

1. 전체 구조

Spring Boot에서 데이터베이스가 바뀌더라도 Entity → Repository → Service → DTO → Controller라는 기본 구조는 거의 동일함.

Client → REST Controller → Service → DTO ↔ Entity → Repository → Spring Data JPA/Hibernate → Oracle · PostgreSQL · MariaDB

2. Oracle · PostgreSQL · MariaDB 비교

항목	Oracle	PostgreSQL	MariaDB
성격	기업용 상용 DB 중심	강력한 오픈소스 DB	MySQL 계열 오픈소스 DB
Spring Boot/JPA	매우 좋음	매우 좋음	매우 좋음
자동 증가	Identity / Sequence	Identity / Sequence	AUTO_INCREMENT
JSON	지원	매우 강력	지원
비용	상용 환경 라이선스 주의	무료	무료
기업/공공	강점	사용 증가	사용
웹 서비스	좋음	매우 좋음	매우 편함
AI/데이터	좋음	추천	좋음
초보 학습	보통	보통	쉬움

3. 권장 프로젝트 구조

```
src/main/
├── java 또는 kotlin/com/example/demo/
│   ├── controller/MemberController
│   ├── service/MemberService
│   ├── repository/MemberRepository
│   ├── entity/Member
│   ├── dto/MemberDto
│   └── DemoApplication
└── resources/application.yml
```

4. Entity

Java

```java
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

Kotlin

```kotlin
@Entity
@Table(name = "member")
class Member(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null,
    var name: String = "",
    var email: String = "",
    var age: Int = 0
)
```

Entity는 데이터베이스 테이블과 매핑되는 객체임.

5. DTO

Java

```java
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class MemberDto {
    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

Kotlin

```kotlin
data class MemberDto(
    val id: Long? = null,
    val name: String,
    val email: String,
    val age: Int
)
```

DTO는 API 계층에서 데이터를 주고받기 위한 객체임. Entity를 API에 직접 노출하지 않는 구조를 만들 때 유용함.

6. Repository

Java

```java
public interface MemberRepository
        extends JpaRepository<Member, Long> {
    List<Member> findByName(String name);
}
```

Kotlin

```kotlin
interface MemberRepository : JpaRepository<Member, Long> {
    fun findByName(name: String): List<Member>
}
```

JpaRepository를 상속하면 save(), findById(), findAll(), delete(), deleteById(), count() 등의 기본 CRUD 기능을 바로 사용할 수 있음.

7. Service - DTO ↔ Entity 변환

Java

```java
@Service
@RequiredArgsConstructor
public class MemberService {
    private final MemberRepository repository;

    public MemberDto save(MemberDto dto) {
        Member member = new Member();
        member.setName(dto.getName());
        member.setEmail(dto.getEmail());
        member.setAge(dto.getAge());

        Member saved = repository.save(member);
        return new MemberDto(saved.getId(), saved.getName(),
                saved.getEmail(), saved.getAge());
    }

    public List<MemberDto> findAll() {
        return repository.findAll().stream()
            .map(m -> new MemberDto(m.getId(), m.getName(),
                    m.getEmail(), m.getAge()))
            .toList();
    }
}
```

Kotlin

```kotlin
@Service
class MemberService(
    private val repository: MemberRepository
) {
    fun save(dto: MemberDto): MemberDto {
        val saved = repository.save(
            Member(name = dto.name, email = dto.email, age = dto.age)
        )
        return MemberDto(saved.id, saved.name, saved.email, saved.age)
    }

    fun findAll(): List<MemberDto> =
        repository.findAll().map {
            MemberDto(it.id, it.name, it.email, it.age)
        }
}
```

8. REST Controller

Java

```java
@RestController
@RequestMapping("/api/members")
@RequiredArgsConstructor
public class MemberController {
    private final MemberService service;

    @PostMapping
    public MemberDto save(@RequestBody MemberDto dto) {
        return service.save(dto);
    }

    @GetMapping
    public List<MemberDto> findAll() {
        return service.findAll();
    }
}
```

Kotlin

```kotlin
@RestController
@RequestMapping("/api/members")
class MemberController(
    private val service: MemberService
) {
    @PostMapping
    fun save(@RequestBody dto: MemberDto) = service.save(dto)

    @GetMapping
    fun findAll() = service.findAll()
}
```

등록 요청 JSON

```text
POST /api/members

{
  "name": "홍길동",
  "email": "hong@test.com",
  "age": 35
}
```

9. 데이터베이스 연결 설정

MariaDB - application.yml

```yaml
spring:
  datasource:
    url: jdbc:mariadb://localhost:3306/testdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

PostgreSQL - application.yml

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/testdb
    username: postgres
    password: 1234
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

Oracle - application.yml

```yaml
spring:
  datasource:
    url: jdbc:oracle:thin:@localhost:1521/FREEPDB1
    username: testuser
    password: 1234
    driver-class-name: oracle.jdbc.OracleDriver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

Gradle JDBC 드라이버 예

```text
// MariaDB
runtimeOnly("org.mariadb.jdbc:mariadb-java-client")

// PostgreSQL
runtimeOnly("org.postgresql:postgresql")

// Oracle
runtimeOnly("com.oracle.database.jdbc:ojdbc11")
```

Oracle의 키 생성 전략은 실제 Oracle 버전과 테이블 설계에 따라 IDENTITY 또는 SEQUENCE를 선택함.

10. Java와 Kotlin 비교표

항목	Spring Boot Java	Spring Boot Kotlin
코드량	상대적으로 많음	적음
DTO	일반 클래스/record/Lombok	data class가 편리
Getter/Setter	직접 작성 또는 Lombok	프로퍼티로 간결
Null 처리	주의 필요	언어 차원에서 강함
Repository	간단	더 간결
Controller	간단	더 간결
JPA 생태계	매우 성숙	매우 좋음
학습자료	매우 많음	많음
기존 기업 프로젝트	강점	증가 추세
신규 개발	좋음	추천

11. 핵심 정리

REST API 정석 구조: Controller → Service → DTO ↔ Entity → Repository → JPA → Database

DB를 MariaDB에서 PostgreSQL 또는 Oracle로 변경해도 애플리케이션 계층 구조는 대부분 유지할 수 있음. JDBC 드라이버와 datasource 설정, 키 생성 방식, DB 고유 SQL/데이터 타입 등의 차이를 주로 확인하면 됨.

학습 순서는 MariaDB로 CRUD를 완성한 뒤 PostgreSQL과 Oracle로 DB만 교체해 동일 API를 실행해 보는 방식이 이해하기 쉬움.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.